```
import re, nltk, spacy, torch
import torch.nn as nn
import numpy as np

from nltk import word_tokenize, pos_tag, CFG, PCFG, ChartParser,
ViterbiParser
from nltk.stem import PorterStemmer, WordNetLemmatizer
from nltk.lm import MLE, Laplace, Lidstone
from nltk.lm.preprocessing import padded_everygram_pipeline
from nltk.corpus import wordnet
from nltk.translate.bleu_score import sentence_bleu, SmoothingFunction

from sklearn.feature_extraction.text import CountVectorizer,
TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity,
euclidean_distances
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score, precision_score,
recall_score, f1_score

from gensim.models import Word2Vec
from transformers import pipeline, AutoTokenizer, AutoModel

nltk.download(["punkt", "punkt_tab", "averaged_perceptron_tagger_eng",
               "wordnet", "omw-1.4"], quiet=True)

nlp = spacy.load("en_core_web_sm")

# COMMON DATASET: F1-F20

new_sentences = [
    "The hotel was clean and comfortable.",
    "The staff were friendly and helpful.",
    "The room was quiet and peaceful.",
    "The food was excellent and fresh.",
    "Ravi liked the service.",
    "The hotel was affordable.",
    "The location was beautiful.",
    "The employees were polite.",
    "The room was dirty and noisy.",
```

```
        "The food was cold and awful.",
        "The service was slow and poor.",
        "The staff were rude and unhelpful.",
        "The hotel was expensive and uncomfortable.",
        "The bathroom was dirty.",
        "The stay was wonderful and relaxing.",
        "The hotel disappointed the visitors.",
        "Google recommended the hotel.",
        "Ravi visited Chennai last week.",
        "He enjoyed the peaceful location.",
        "The hotel provided excellent service."
    ]

    sentiment_labels = [
        1, 1, 1, 1, 1, 1, 1, 1, 0, 0,
        0, 0, 0, 0, 1, 0, 1, 1, 1, 1
    ]

    # UNIT I: TEXT PREPROCESSING

    stemmer = PorterStemmer()
    lemmatizer = WordNetLemmatizer()

    for i, sentence in enumerate(new_sentences):
        text = sentence.lower()
        text = re.sub(r"http\S+|[^a-zA-Z\s]", "", text)
        tokens = word_tokenize(text)

        print(f"\nF{i+1}: {sentence}")
        print("Tokens:", tokens)
        print("Stems:", [stemmer.stem(w) for w in tokens])
        print("Lemmas:", [lemmatizer.lemmatize(w, pos="v") for w in tokens])

    # TOKENIZE ALL SENTENCES

    tokenized = [
        word_tokenize(re.sub(r"http\S+|[^a-zA-Z\s]", "", s.lower()))
        for s in new_sentences
    ]

    # N-GRAM LANGUAGE MODELS AND SMOOTHING
```

```
train_mle, vocab = padded_everygram_pipeline(2, tokenized)
mle = MLE(2)
mle.fit(train_mle, vocab)

train_lap, vocab = padded_everygram_pipeline(2, tokenized)
laplace = Laplace(2)
laplace.fit(train_lap, vocab)

train_lid, vocab = padded_everygram_pipeline(2, tokenized)
lidstone = Lidstone(0.5, 2)
lidstone.fit(train_lid, vocab)

print("\nMLE P(hotel | the):", mle.score("hotel", ["the"]))
print("Laplace P(hotel | the):", laplace.score("hotel", ["the"]))
print("Lidstone P(hotel | the):", lidstone.score("hotel", ["the"]))

# NEURAL LANGUAGE MODEL: NEXT-WORD PREDICTION

words = "the hotel was clean and comfortable".split()
w2i = {w: i for i, w in enumerate(words)}

X = torch.tensor([w2i[w] for w in words[:-1]])
y = torch.tensor([w2i[w] for w in words[1:]])

model = nn.Sequential(
    nn.Embedding(len(words), 8),
    nn.Linear(8, len(words))
)

opt = torch.optim.Adam(model.parameters(), lr=0.05)
loss_fn = nn.CrossEntropyLoss()

for _ in range(50):
    opt.zero_grad()
    loss = loss_fn(model(X), y)
    loss.backward()
    opt.step()

print("Predicted Next Words:",
      [words[i] for i in model(X).argmax(dim=-1).tolist()])
```

```
# UNIT II: POS TAGGING - ALL SENTENCES

for i, sentence in enumerate(new_sentences):
    print(f"\nF{i+1} POS Tags:", pos_tag(word_tokenize(sentence)))

# CONTEXT-FREE GRAMMAR (CFG)

cfg = CFG.fromstring("""
S -> NP VP
NP -> Det N | Det N PP
VP -> V NP | VP PP
PP -> P NP
Det -> 'the'
N -> 'boy' | 'man' | 'telescope'
V -> 'saw'
P -> 'with'
""")

parser = ChartParser(cfg)
sent = "the boy saw the man with the telescope".split()

for tree in parser.parse(sent):
    tree.pretty_print()

# PROBABILISTIC CFG (PCFG)

pcfg = PCFG.fromstring("""
S -> NP VP [1.0]
NP -> Det N [0.6] | Det N PP [0.4]
VP -> V NP [0.7] | VP PP [0.3]
PP -> P NP [1.0]
Det -> 'the' [1.0]
N -> 'boy' [0.4] | 'man' [0.3] | 'telescope' [0.3]
V -> 'saw' [1.0]
P -> 'with' [1.0]
""")

viterbi = ViterbiParser(pcfg)

for tree in viterbi.parse(sent):
```

```
        tree.pretty_print()

# DEPENDENCY PARSING - ALL SENTENCES

for i, sentence in enumerate(new_sentences):
    doc = nlp(sentence)
    print(f"\nF{i+1} Dependency Parsing:")
    for token in doc:
        print(f"{token.text:12} --({token.dep_:8})-->
{token.head.text}")

# UNIT III: WORDNET

for word in ["hotel", "service", "food"]:
    synsets = wordnet.synsets(word)
    if len(synsets) >= 2:
        print(word, synsets[0].definition())
        print("Similarity:", synsets[0].wup_similarity(synsets[1]))

# VECTOR SPACE MODELS: BOW AND TF-IDF

bow_vectorizer = CountVectorizer()
bow = bow_vectorizer.fit_transform(new_sentences)

tfidf_vectorizer = TfidfVectorizer()
tfidf = tfidf_vectorizer.fit_transform(new_sentences)

print("\nBoW Shape:", bow.shape)
print("TF-IDF Shape:", tfidf.shape)
print("First TF-IDF Row:", tfidf.toarray()[0].round(2))

# SIMILARITY MEASURES - ALL SENTENCE PAIRS

cos_sim = cosine_similarity(tfidf)
euc_dist = euclidean_distances(tfidf)

print("F1 vs F2 Cosine Similarity:", round(cos_sim[0, 1], 3))
print("F1 vs F2 Euclidean Distance:", round(euc_dist[0, 1], 3))

# WORD EMBEDDINGS: WORD2VEC
```

```
w2v = Word2Vec(
    tokenized, vector_size=20, window=2,
    min_count=1, sg=1, seed=42
)

for word in ["hotel", "service", "food"]:
    if word in w2v.wv:
        print(f"{word} Vector:", w2v.wv[word][:5].round(3))
        print("Similar words:", w2v.wv.most_similar(word, topn=2))

# TEXT CLASSIFICATION: NAIVE BAYES

vec = TfidfVectorizer()
X_vec = vec.fit_transform(new_sentences)

clf = MultinomialNB()
clf.fit(X_vec, sentiment_labels)

pred = clf.predict(vec.transform([
    "The hotel was excellent and clean.",
    "The room was dirty and noisy."
]))

print("\nText Classification Predictions:",
      ["Positive" if p == 1 else "Negative" for p in pred])

# UNIT IV: COREFERENCE RESOLUTION - ALL SENTENCES

for i, sentence in enumerate(new_sentences):
    doc = nlp(sentence)
    persons = [e.text for e in doc.ents if e.label_ == "PERSON"]

    for token in doc:
        if token.text.lower() in ["he", "she", "him", "her"] and
persons:
            print(f"F{i+1}: {token.text} may refer to {persons[-1]}")

# RNN-BASED MODEL

rnn = nn.RNN(input_size=8, hidden_size=16, batch_first=True)
fc = nn.Linear(16, 2)
```

```
sample_input = torch.randn(len(new_sentences), 5, 8)
output, hidden = rnn(sample_input)
logits = fc(hidden[-1])

print("\nRNN Output Shape:", logits.shape)

# MACHINE TRANSLATION - ALL SENTENCES

translator = pipeline(
    "translation_en_to_fr",
    model="Helsinki-NLP/opus-mt-en-fr"
)

for i, sentence in enumerate(new_sentences):
    print(f"F{i+1} Translation:",
          translator(sentence, max_length=100)[0]["translation_text"])

# EVALUATION METRICS: BLEU

ref = [["the", "hotel", "was", "clean", "and", "comfortable"]]
cand = word_tokenize(new_sentences[0].lower())

print("\nBLEU Score:", sentence_bleu(
    ref, cand, smoothing_function=SmoothingFunction().method1
))

# CLASSIFICATION METRICS
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    new_sentences, sentiment_labels,
    test_size=0.2, random_state=42, stratify=sentiment_labels
)

vec = TfidfVectorizer()
X_train = vec.fit_transform(X_train)
X_test = vec.transform(X_test)

clf = MultinomialNB()
clf.fit(X_train, y_train)
```

```
y_pred = clf.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred, zero_division=0))
print("Recall:", recall_score(y_test, y_pred, zero_division=0))
print("F1 Score:", f1_score(y_test, y_pred, zero_division=0))

# UNIT V: ATTENTION MECHANISM - ALL SENTENCES
# Demonstration with random query/key/value vectors.

Q = torch.randn(len(new_sentences), 4)
K = torch.randn(len(new_sentences), 4)
V = torch.randn(len(new_sentences), 4)

scores = (Q @ K.T) / np.sqrt(4)
weights = torch.softmax(scores, dim=-1)
attention_out = weights @ V

print("\nAttention Weights Shape:", weights.shape)
print("Attention Output Shape:", attention_out.shape)

# TRANSFORMER SENTIMENT ANALYSIS - ALL SENTENCES

sentiment_pipeline = pipeline("sentiment-analysis")

for i, sentence in enumerate(new_sentences):
    print(f"F{i+1} Transformer Sentiment:",
          sentiment_pipeline(sentence))

# BERT FILL-MASK - ALL SENTENCES
# Replace one word with [MASK] to form a valid BERT input.

fill = pipeline("fill-mask", model="bert-base-uncased")

masked_sentences = [
    "The hotel was [MASK] and comfortable.",
    "The staff were [MASK] and helpful.",
    "The room was [MASK] and noisy.",
    "The food was [MASK] and awful.",
    "The service was [MASK] and poor."
```

```
]

for sentence in masked_sentences:
    print("BERT Fill-Mask:", fill(sentence)[0]["sequence"])

# BERT CONTEXTUAL EMBEDDINGS - ALL SENTENCES

tok = AutoTokenizer.from_pretrained("bert-base-uncased")
bert = AutoModel.from_pretrained("bert-base-uncased")
bert.eval()

with torch.no_grad():
    for i, sentence in enumerate(new_sentences):
        inputs = tok(sentence, return_tensors="pt")
        emb = bert(**inputs).last_hidden_state
        print(f"F{i+1} BERT Embedding Shape:", emb.shape)

# NAMED ENTITY RECOGNITION (NER) - ALL SENTENCES

for i, sentence in enumerate(new_sentences):
    doc = nlp(sentence)
    print(f"\nF{i+1} Entities:")
    for ent in doc.ents:
        print(f"Entity: {ent.text:12} | Label: {ent.label_}")
```